

# Honors Class03 Activity: PEMDAS and Modules

Team Members: \_\_\_\_\_

---

## Learning Objectives

- Order of operations in Python (PEMDAS)
- Using functions from modules
- Turning word problems into code

## Part 1: PEMDAS in Python

In Python, the order of operations follows the PEMDAS rule:

- Parentheses
- Exponents
- Multiplication and Division (from left to right)
- Addition and Subtraction (from left to right)

Create a Jupyter notebook and code the following expressions in Python to find the solutions. Check with your teammates that you are getting the same result:

1.  $(4^2 - 5 \times 3) \div (12 \div 3) + 7 =$  \_\_\_\_\_

2.  $3^3 - [2 \times (14 - 3^2)] \div 4 =$  \_\_\_\_\_

3.  $\frac{(5^3 - 2^4) \div (18 \div 3) + 4^2}{(7 - 3)^2} =$  \_\_\_\_\_

4.  $6^2 - \left[ \frac{3 \times (2^5 - 4^2)}{(9 - 3)^2} \right] =$  \_\_\_\_\_

5.  $\left( \frac{4^3 + 2^5}{3^2} - 5 \right) \div \left( 1 + \frac{8}{2^3} \right) =$  \_\_\_\_\_

## Part 2: Using Modules in Python

### Built-in Python Functions

Many math functions are built into the Python programming language.

Create a new Jupyter notebook and try entering the following code:

```
maximum_number = max(3, 4.4, 12, 0, -8)
maximum_number
```

You are calling the Python function `max()` and assigning the result to the variable `maximum_number`. Then, when you type the variable name on a separate line the result is displayed. Naturally, there is also a `min()` function.

Try `min()` with the same set of numbers.

## External Modules

To keep Python lean, not every function is baked into the language. Often you must load the functions you need from an external module. Here we import the `math` module so we have access to the value `pi`. Try this:

```
import math
print(math.pi)
```

After you have imported the `math` module, you have access to all of its functions using dot notation. Thus, `math.pi` gives you the value for `pi` from the `math` module.

Use `math.pi` to find the area of a circle with a radius of 4: \_\_\_\_\_

The `math` module contains many functions. To see the complete list, type `dir(math)` in a code cell that comes after you have imported the `math` module.

There are many, many modules available in Python. Each module is a library of functions designed to assist with a particular task — math problems, machine learning, plotting graphs, working with databases, designing websites, creating games — you name it, there is probably a Python module that does it! Often, more than one. You will become familiar with quite a few Python modules in this course.

## Turning Word Problems into Code

Learning to program is all about breaking down a problem into simple steps that can be expressed in code. Computers do precisely what you tell them to do — no more, no less. You need to know exactly what steps are needed before you can create a program that solves a problem. Your instructions have to be clear and free of typos. Let's code some simple word problems for practice.

### **Problem**

A stream's water level was measured at four times during the day: 1.12, 1.18, 1.09, 1.15 meters. What is the average water level? \_\_\_\_\_

### **Problem**

A pond's water level increased from 0.85 m to 0.92 m after rainfall. What is the percent increase?  
\_\_\_\_\_

### **Problem**

A student's final grade is:

- Homework: 30% (92)
- Midterm: 25% (85)
- Final: 45% (88)

What is the final numeric grade? \_\_\_\_\_

## Breaking Down Longer Problems

As problems become more complicated, it is important to solve them in steps, creating Python variables to hold intermediate values.

### **Problem**

An environmental science class is planning a field trip.

- The bus costs \$485 total.
- Each student ticket costs \$12.50.
- There are 28 students and 2 instructors, and instructors do not pay ticket fees.

The department will subsidize 40% of the total cost.

**What is the cost per student after the subsidy? Let's work an example together.**

```
bus_cost = 485
ticket_cost = 12.5
num_students = 28
subsidy = 0.4

total_cost = bus_cost + ticket_cost * num_students
print("The total cost is:", total_cost)

cost_with_subsidy = total_cost - total_cost * subsidy
print("Cost after subsidy is:", cost_with_subsidy)

cost_per_student = cost_with_subsidy / num_students
print("Cost per student is:", cost_per_student)
```

### **Key points**

- Print and check intermediate results. It is easier to debug your code when the final answer is not what you expected.
- Pick variable names that are easy to understand.

Once a solution is programmed this way, it is easy to change a value.

**Suppose the department decides to increase the subsidy to 50%. What is the new price per student?**

---

Now it is your turn. Create as many code cells as you need.

### **Problem**

A rain gauge recorded rainfall over three consecutive intervals:

- 18 mm in 30 minutes
- 22 mm in 45 minutes
- 10 mm in 15 minutes

**What was the total rainfall?**

**What was the average rainfall rate (mm/hr) over the entire storm?**

### **Problem**

A field crew orders sampling supplies:

- 6 test kits at \$18.75 each
- 3 calibration solutions at \$42.00 each

Shipping costs \$24.50. Sales tax is 7.25% and applies only to the supplies, not shipping. After everything is purchased, the vendor issues a \$35 rebate.

**What is the final total cost?**

### **Reflection**

Each team member solved at least one problem using intermediate variables.

- Choose one problem you all attempted.
- How did your variable choices and variable names differ?
- Did one approach feel clearer or easier to debug? Why?